

involved are as follows:

1. Connect to server
2. Log in
3. Make service request to perform certain operations
4. Quit

When using Python's FTP support, all you do is import the *ftplib* module and instantiate the *ftplib.FTP* class. All FTP activity—logging in, transferring files and logging out—will be accomplished using your object. Here is some Python pseudocode:

```
from ftplib import FTP
f = FTP('some.ftp.server')
f.login('username', 'password')
f.quit()
```

The FTP class and methods

Let us take a look at the *ftplib.FTP* class that we are going to use in our code. Table 1 lists most of the methods and their applications available in this class.



Note 1: For demonstration purposes, I have installed an FTP server on my local machine and am using my localhost as the *ftpserver*.

Here is an example:

```
>>> from ftplib import FTP
>>> f = FTP('127.0.0.1')
>>> f.login('username', 'password****')
'230 Guest login ok, access restrictions apply.'
>>> f.dir()
drwxr-xr-x  2 1000   1000   4096 Nov 13 21:40 Desktop
drwxr-xr-x  7 1000   1000   4096 Nov 24 21:24 Documents
drwxr-xr-x  2 1000   1000   20480 Nov 27 18:59 Downloads
drwxr-xr-x  2 1000   1000   4096 Nov 24 21:15 Music
drwxr-xr-x  3 1000   1000   4096 Nov 22 22:02 Pictures
drwxr-xr-x  2 1000   1000   4096 Oct 13 15:15 Public
-rwxr-xr-x  1 1000   1000   952 Nov 19 18:41 net.py
-rw-rw-r--  1 1000   1000   860 Nov 15 13:49 time.py
>>> f.pwd()
'/home/christmas'
>>> f.cwd('/var')
'250 Directory successfully changed.'
>>> f.pwd()
'/var'
```

Table 1

Methods for FTP objects

Method	Description
<i>login</i> (user='anonymous', passwd="", acct=")	Log in to FTP server; all arguments are optional
<i>pwd</i> ()	Current working directory
<i>cwd</i> (path)	Change current working directory to path
<i>dir</i> ([path[,...[,cb]])	Displays directory listing of path. Optional call-back cb passed to <i>retrlines</i> ()
<i>nlist</i> ([path[,...]])	Like <i>dir</i> () but returns a list of filenames instead of displaying them
<i>retrlines</i> (cmd [, cb])	Downloads text file using given FTP cmd, for example, RETR filename; option callback cb is used for processing each line of file
<i>retrbinary</i> (cmd,cb[, bs=8192[, ra]])	Similar to <i>retrlines</i> () except for the binary file; callback cb for processing each block (default block size is 8K)
<i>storlines</i> (cmd, f)	Uploads text file using given FTP cmd - for example, STOR file name
<i>storbinary</i> (cmd,f[, bs=8192])	Similar to <i>storlines</i> () but is used for binary files
<i>delete</i> (path)	Deletes remote file located at path
<i>mkd</i> (directory)	Creates remote directory
<i>rmd</i> (directory)	Removes remote directory
<i>exception ftplib.error_reply</i>	Exception raised when an unexpected reply is received from the server.
<i>exception ftplib.error_temp</i>	Exception raised when an error code signifying a temporary error (response codes in the range 400–499) is received.
<i>exception ftplib.error_perm</i>	Exception raised when an error code signifying a permanent error (response codes in the range 500–599) is received.
<i>exception ftplib.error_proto</i>	Exception raised when a reply is received from the server that does not fit the response specifications of the FTP, i.e., begins with a digit in the range 1–5.
<i>ftplib.all_errors</i>	The set of all exceptions (as a tuple) that methods of FTP instances may raise as a result of problems with the FTP connection (as opposed to programming errors made by the caller). This set includes the four exceptions listed above as well as <i>socket.error</i> and <i>IOError</i> .
<i>connect</i> (host[, port[, timeout]])	Connects to the given host and port. The default port number is 21, as specified by the FTP protocol.
<i>quit</i> ()	Closes connection and quits.